МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине

«Облачные и туманные вычисления»

Выполнил:
Студент группы P3411
Болорболд Аригуун

Преподаватель:
Перл Ольга Вячеславовна

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Программная реализация 4
   2.2. Инфраструктура 9
   2.3. Настройка `Node-RED` 12
3. Вывод 15

1. Текст задания

Цель:

Смоделировать среду `Edge/Fog`, где данные обрабатываются локально (на «туманном» уровне) перед отправкой в Облако. Это критически важно для вашего курса, так как демонстрирует работу шлюзов (`Gateways`).

Сценарий: На заводе установлены датчики температуры. Нужен «Туманный узел» (Шлюз), который фильтрует нормальные показатели, но немедленно оповещает Облако, если температура превышает норму.

Архитектура (все компоненты запускаются в `Docker`):

- **Edge Device (Датчик)**: Простой скрипт (контейнер `Python`), генерирующий случайную температуру и публикующий её в топик `MQTT` каждую секунду.
- **Fog Node (Шлюз)**: Контейнер с `Node-RED`, подписывающийся на данные датчика и реализующий фильтрацию. Логика: Если Темп < 80, ничего не делать (или писать в локальный лог). Если Темп > 80, переслать в Облако.
- **Cloud (Центр)**: Контейнер `Mosquitto` (`MQTT` брокер) или простой дашборд, принимающий только критические алерты.

Ход работы:

1. Создать специализированную сеть в `Docker (factory-net)`.
2. Открыть визуальный редактор `Node-RED` (`localhost:1880`) и собрать логику: вход `MQTT` → `Switch Node` (если > 80) → выход `MQTT` (топик Облака).
3. Запустить контейнер «Датчика».
4. Наблюдать, что контейнер «Облако» получает лишь критический трафик, демонстрируя экономию пропускной способности.

Ключевые знания: `MQTT`, фильтрация на краю (`Edge Filtering`), оптимизация трафика, логика работы шлюзов.

2. Выполнение

2.1. Программная реализация

2.1.1. Сенсор

Скрипт представляет из себя имитацию работу датчика, отправляя случайные значения температуры в формате `JSON` в топик `factory/temp` локального брокера:

```
import time
import random
import json
import paho.mqtt.client as mqtt
import os

BROKER = os.getenv("MQTT_BROKER", "edge-broker")
PORT = int(os.getenv("MQTT_PORT", 1883))
TOPIC = os.getenv("MQTT_TOPIC", "factory/temp")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

def connect():
    while True:
        try:
            client.connect(BROKER, PORT, 60)
            print(f"Connected to {BROKER}:{PORT}")
            return
        except Exception as e:
            print(f"Connection failed: {e}. Retrying in 5s...")
            time.sleep(5)

connect()

print(f"Publishing temperature data to topic: {TOPIC}")
while True:
    temp = random.randint(20, 100)
    payload = json.dumps({"temp": temp, "ts": time.time()})
    client.publish(TOPIC, payload)
    print(f"Sent: {payload} to {TOPIC}")
    time.sleep(1)
```

2.1.2. Облачная панель

Этот скрипт подписывается на топик `cloud/alerts` облачного брокера и выводит полученные сообщения (алерты) в консоль.

```python
import paho.mqtt.client as mqtt
import os
import json
from flask import Flask, render_template_string
from threading import Thread
import time
from collections import deque

BROKER = os.getenv("MQTT_BROKER", "cloud-broker")
PORT = int(os.getenv("MQTT_PORT", 1883))
TOPIC = os.getenv("MQTT_TOPIC", "cloud/alerts")

app = Flask(__name__)

alerts = deque(maxlen=50)
stats = {
    'total_alerts': 0,
    'last_alert_time': None,
    'avg_temp': 0,
    'max_temp': 0
}

HTML_TEMPLATE = """
# здесь простой HTML/CSS
"""
@app.route("/")
def dashboard():
    return render_template_string(HTML_TEMPLATE,
alerts=list(alerts)[::-1], stats=stats)

def on_connect(client, userdata, flags, reason_code, properties):
    client.subscribe(TOPIC)

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        temp = data.get('temp')
        ts = data.get('ts')

        stats['total_alerts'] += 1
        stats['last_alert_time'] = time.strftime('%H:%M:%S',
time.localtime(ts))

        temps = [a['temp'] for a in alerts] + [temp]
        stats['avg_temp'] = sum(temps) / len(temps)
        stats['max_temp'] = max(temps)
```

```
        alerts.append({
            'temp': temp,
            'time': time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(ts))
        })
        print(f"ALERT RECEIVED: Temp={temp}°C at timestamp={ts}")
    except Exception as e:
        print(f"Error processing message: {e}")


def mqtt_loop():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect
    client.on_message = on_message
    print(f"Connecting to {BROKER}:{PORT}...")
    client.connect(BROKER, PORT, 60)
    client.loop_forever()


if __name__ == "__main__":
    mqtt_thread = Thread(target=mqtt_loop, daemon=True)
    mqtt_thread.start()
    app.run(host="0.0.0.0", port=5001)
```

2.1.3. Сравнительная панель

Этот скрипт подписывается на топик `cloud/alerts` облачного брокера и выводит полученные сообщения (алерты) в консоль.

```
import paho.mqtt.client as mqtt
import json
from flask import Flask, render_template_string
from threading import Thread
import time
from collections import deque

app = Flask(__name__)

sensor_data = deque(maxlen=100)
cloud_data = deque(maxlen=100)

stats = {
    'total_sensor': 0,
    'total_cloud': 0,
    'bandwidth_saved': 0,
    'avg_sensor_temp': 0,
```

```
    'avg_cloud_temp': 0
}

HTML_TEMPLATE = """
# HTML/CSS
"""

@app.route("/")
def dashboard():
    return render_template_string(
        HTML_TEMPLATE,
        sensor_data=list(sensor_data)[::-1],
        cloud_data=list(cloud_data)[::-1],
        stats=stats
    )


def on_connect_sensor(client, userdata, flags, reason_code,
properties):
    print(f"Connected to Edge Broker (sensor data)")
    client.subscribe("factory/temp")


def on_message_sensor(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        temp = data.get('temp')
        ts = data.get('ts')

        stats['total_sensor'] += 1

        sensor_temps = [item['temp'] for item in sensor_data] +
[temp]
        stats['avg_sensor_temp'] = sum(sensor_temps) /
len(sensor_temps)

        sensor_data.append({
            'temp': temp,
            'time': time.strftime('%H:%M:%S', time.localtime(ts))
        })

        if stats['total_sensor'] > 0:
            stats['bandwidth_saved'] = ((stats['total_sensor'] -
stats['total_cloud']) / stats['total_sensor']) * 100

    except Exception as e:
```

```
        print(f"Error processing sensor message: {e}")


def on_connect_cloud(client, userdata, flags, reason_code,
properties):
    print(f"Connected to Cloud Broker (filtered data)")
    client.subscribe("cloud/alerts")


def on_message_cloud(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        temp = data.get('temp')
        ts = data.get('ts')

        stats['total_cloud'] += 1

        cloud_temps = [item['temp'] for item in cloud_data] +
[temp]
        if cloud_temps:
            stats['avg_cloud_temp'] = sum(cloud_temps) /
len(cloud_temps)

        cloud_data.append({
            'temp': temp,
            'time': time.strftime('%H:%M:%S', time.localtime(ts))
        })

        if stats['total_sensor'] > 0:
            stats['bandwidth_saved'] = ((stats['total_sensor'] -
stats['total_cloud']) / stats['total_sensor']) * 100

    except Exception as e:
        print(f"Error processing cloud message: {e}")


def mqtt_sensor_loop():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect_sensor
    client.on_message = on_message_sensor
    client.connect("edge-broker", 1883, 60)
    client.loop_forever()


def mqtt_cloud_loop():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
```

```
    client.on_connect = on_connect_cloud
    client.on_message = on_message_cloud
    client.connect("cloud-broker", 1883, 60)
    client.loop_forever()


if __name__ == "__main__":
    sensor_thread = Thread(target=mqtt_sensor_loop, daemon=True)
    cloud_thread = Thread(target=mqtt_cloud_loop, daemon=True)

    sensor_thread.start()
    cloud_thread.start()

    app.run(host="0.0.0.0", port=5002)
```

2.2. Инфраструктура

2.2.1. Конфигурация `Mosquitto`

Для работы брокеров используется конфигурационный файл, разрешающий анонимный доступ и прослушивание порта 1883.

```
listener 1883
allow_anonymous true
```

2.2.2. Docker Compose

В файле `docker-compose.yml` описываются 5 сервисов:

1. `edge-broker`: Локальный `MQTT` брокер.
2. `sensor`: Датчик (генератор данных).
3. `fog-node`: Node-RED для обработки.
4. `cloud-broker`: Облачный `MQTT` брокер.
5. `cloud-dashboard`: Панель мониторинга.
6. `comparison-dashboard`: Панель сравнения всех и критических событий.

```
services:
  edge-broker:
    image: eclipse-mosquitto:2
    container_name: edge-broker
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf
    networks:
      - factory-net

  sensor:
    build: ./sensor
```

```yaml
    container_name: sensor
    environment:
      - MQTT_BROKER=edge-broker
      - MQTT_PORT=1883
      - MQTT_TOPIC=factory/temp
    depends_on:
      - edge-broker
    networks:
      - factory-net
    restart: unless-stopped

  fog-node:
    image: nodered/node-red:latest
    container_name: fog-node
    ports:
      - "1880:1880"
    environment:
      - TZ=Asia/Ulaanbaatar
    volumes:
      - node-red-data:/data
    networks:
      - factory-net
    restart: unless-stopped

  cloud-broker:
    image: eclipse-mosquitto:2
    container_name: cloud-broker
    ports:
      - "1884:1883"
    volumes:
      - ./mosquitto.conf:/mosquitto/config/mosquitto.conf
    networks:
      - factory-net

  cloud-dashboard:
    build: ./cloud_dashboard
    container_name: cloud_dashboard
    environment:
      - MQTT_BROKER=cloud-broker
      - MQTT_PORT=1883
      - MQTT_TOPIC=cloud/alerts
    depends_on:
      - cloud-broker
    networks:
      - factory-net
    restart: unless-stopped
```

```yaml
  comparison-dashboard:
    build: ./comparison_dashboard
    container_name: comparison-dashboard
    ports:
      - "5002:5002"
    depends_on:
      - edge-broker
      - cloud-broker
    networks:
      - factory-net
    restart: unless-stopped

networks:
  factory-net:
    driver: bridge

volumes:
  node-red-data:
```

Если запустить образ, то получится следующее:

```
docker compose ps
```

```
(.venv) PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab3> docker compose ps
NAME              IMAGE                      COMMAND                  SERVICE           CREATED              STATUS                        PORTS
cloud-broker      eclipse-mosquitto:2        "/docker-entrypoint.…"   cloud-broker      2 minutes ago        Up About a minute             0.0.0.0:1884->1883/tcp, [::]:1884->1883/tcp
cloud_dashboard   lab3-cloud-dashboard       "python dashboard.py"    cloud-dashboard   About a minute ago   Up About a minute
edge-broker       eclipse-mosquitto:2        "/docker-entrypoint.…"   edge-broker       2 minutes ago        Up About a minute             0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
fog-node          nodered/node-red:latest    "./entrypoint.sh"        fog-node          2 minutes ago        Up About a minute (healthy)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp
sensor            lab3-sensor                "python sensor.py"       sensor            About a minute ago   Up About a minute
(.venv) PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab3>
```

Рисунок 1 — Процессы в запущенном образе `Docker`

Проверим работу «датчика»:

```
docker compose logs -f sensor
```

```
(.venv) PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab3> docker compose logs -f sensor
sensor  | Connected to edge-broker:1883
sensor  | Publishing temperature data to topic: factory/temp
sensor  | Sent: {"temp": 50, "ts": 1768847272.6924453} to factory/temp
sensor  | Sent: {"temp": 38, "ts": 1768847273.6928444} to factory/temp
sensor  | Sent: {"temp": 21, "ts": 1768847274.6931362} to factory/temp
sensor  | Sent: {"temp": 25, "ts": 1768847275.6934018} to factory/temp
sensor  | Sent: {"temp": 74, "ts": 1768847276.693698} to factory/temp
sensor  | Sent: {"temp": 61, "ts": 1768847277.6940129} to factory/temp
sensor  | Sent: {"temp": 79, "ts": 1768847278.6942935} to factory/temp
sensor  | Sent: {"temp": 89, "ts": 1768847279.6945665} to factory/temp
```

Рисунок 2 — Логи от датчика

2.3. Настройка `Node-RED`

2.3.1. Общий вид потока

Поток состоит из следующих узлов:

1. `MQTT In`: Подписка на топик `factory/temp` от `Edge Broker`.
2. `JSON`: Парсинг входящего сообщения.
3. `Switch`: Фильтрация значений температуры (> 80?).
4. `MQTT Out`: Публикация критических значений в топик `cloud/alerts` `Cloud Broker`.

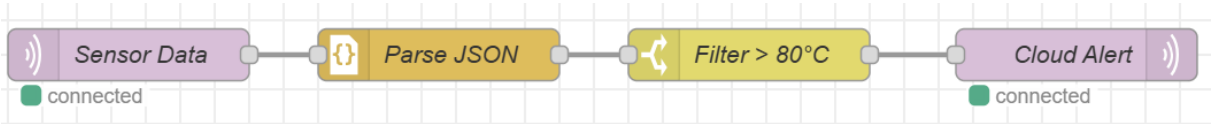

Рисунок 3 — Общий вид потока в `Node-RED`

2.3.2. Настройка узлов

Настройка подключения к локальному брокеру `Edge Broker`.

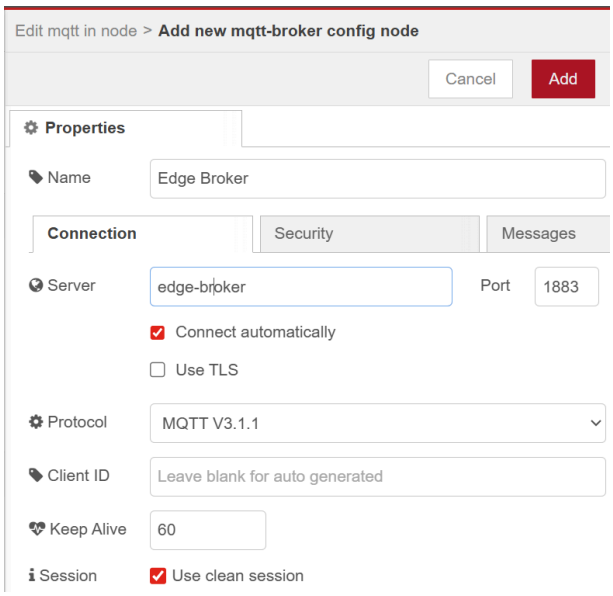

Рисунок 4 — Настройка `Edge Broker`

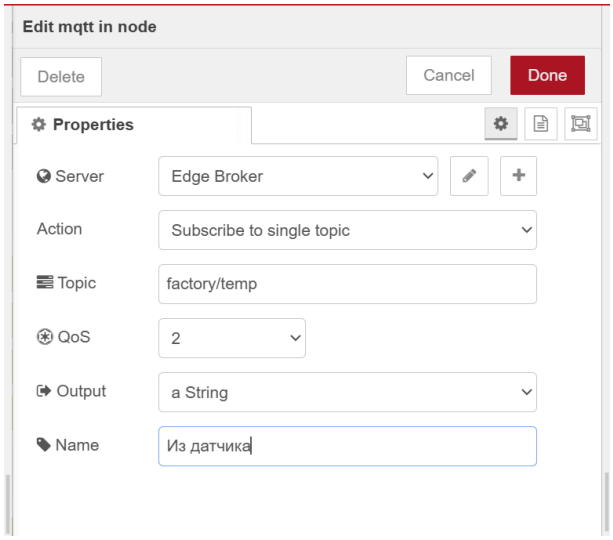

Рисунок 5 — Настройка узла `MQTT In`

Условие фильтрации: пропускать сообщения, где `payload.temp > 80`.

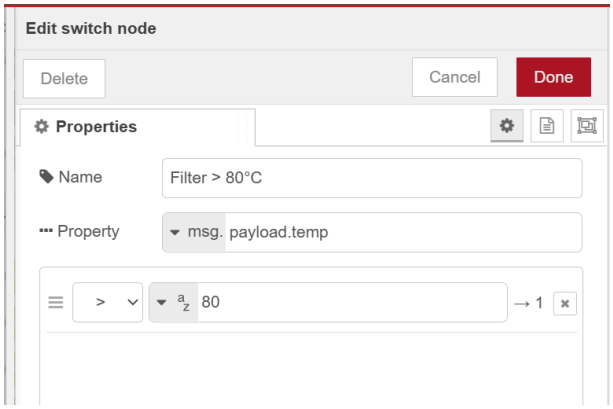

Рисунок 6 — Настройка узла `Switch`

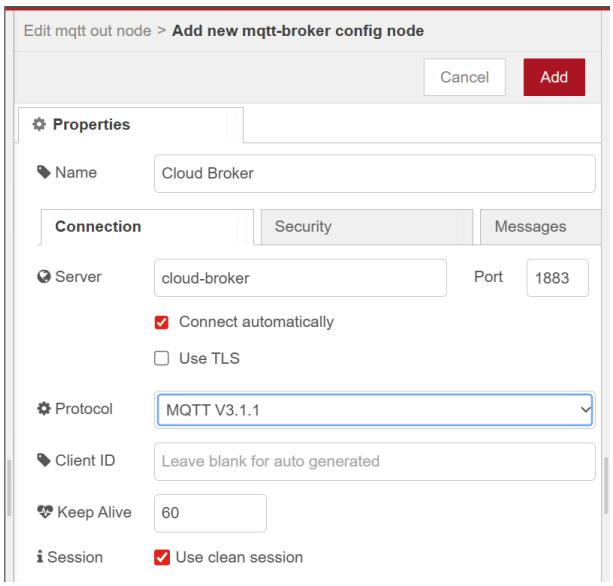

Рисунок 7 — Настройка `Cloud Broker`

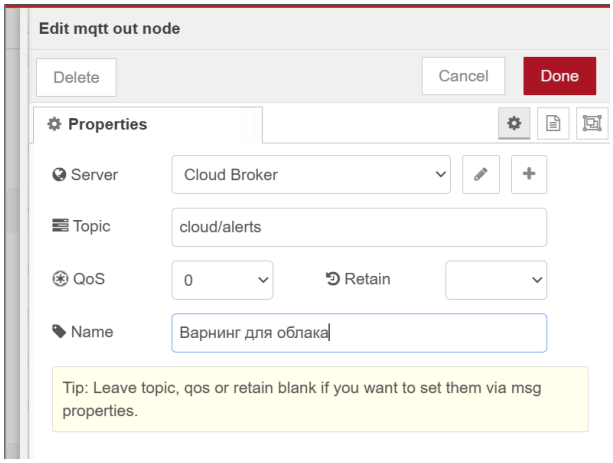

Рисунок 8 — Настройка узла `MQTT Out`

Снизу иллюстрированы выводы всех событий от датчика, а также и критических событий (`> 80°C`):

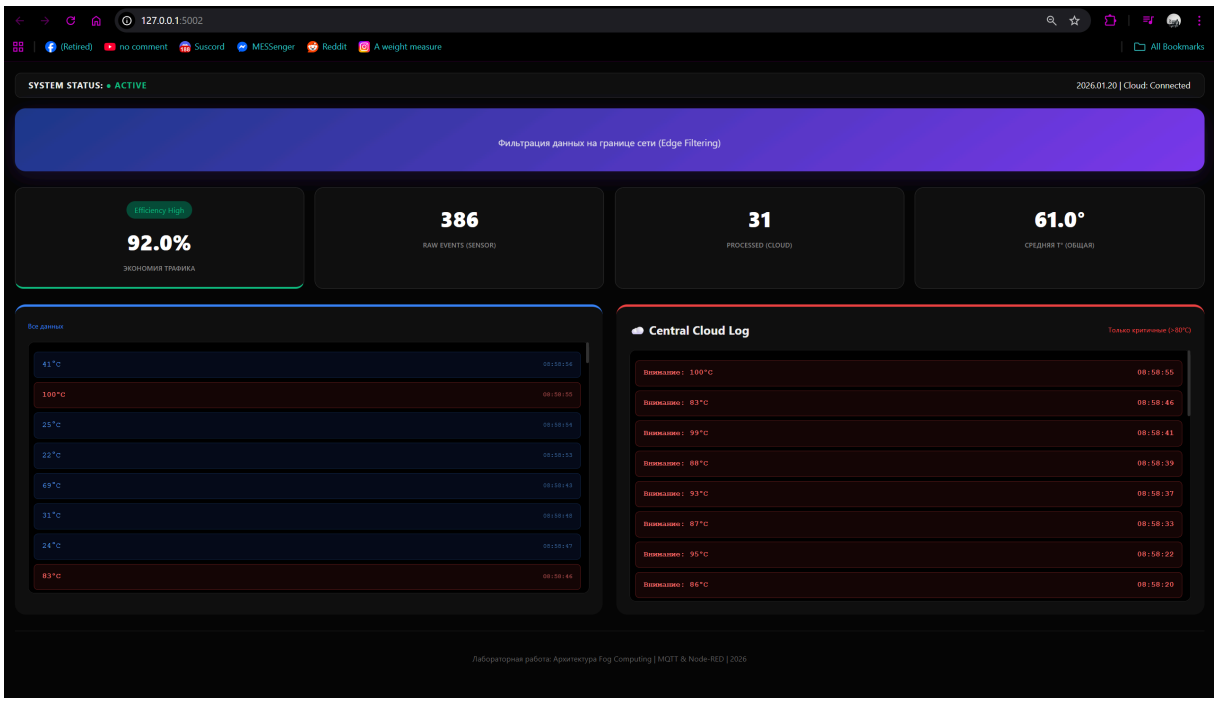

Рисунок 9 — Панель сравнении

3. Вывод

В ходе работы была реализована модель туманных вычислений, ориентированная на оптимизацию потоков данных. С помощью `Node-RED` на уровне `Fog Node` была настроена предварительная обработка сигналов, поступающих по протоколу `MQTT`. Внедрение обработки на периферии позволило отсеивать избыточную информацию, направляя в облако лишь значимые инциденты. Итогом стало существенное снижение нагрузки на каналы связи и вычислительные мощности центрального сегмента системы.